

Installing SD Core

What the installer does to the machine, the two kinds of installation, and the choices it puts in front of you.

There is no compiler to run and no dependency to resolve. SD Core for Windows ships as a single `sd-setup-w1.0-0.exe`, carries its own runtime beside `sd.exe`, and installs in one pass.

This is the largest single difference from the regular SD version — that, and the fact that this version is for Windows and not Linux. In regular SD, `installsdai.sh` existed because ScarletDME targeted four distributions and the end user had to compile; Windows has one target and one ABI, so none of that transfers.

Before you start

You need an elevated session. The installer creates local groups, sets file permissions, registers a service and assigns user rights. You can start the install from a normal console session — you will get an elevation prompt.

SD cannot be installed silently. This is deliberate, not a missing feature. The installer asks questions whose answers cannot be defaulted safely — which kind of installation, whether to expose ports, and the password on the account it makes for you.

The installer does not ask where to put SD. Both roots are fixed. See What lands where.

IT CHECKS THE MACHINE BEFORE IT CHANGES ANYTHING

The installer refuses to start, and changes nothing, if it finds:

- another ssh server using port 22, or
- an ssh service that is not part of Windows, installed even if it is not running, or
- Windows' own ssh server with settings somebody has already changed.

It tells you what it finds — the service and its path, or the directive that was added — so you can decide what to do. You may **remove the other ssh server, or put the ssh configuration back the way Windows shipped it, and run the installer again.**

Why it is this strict: accounts SD creates cannot log in to Windows at all, so ssh is one of only two ways they reach the machine — and SD configures the ssh server so those sessions land inside SD and cannot reach a command prompt. **SD can only promise that about a server it installed and configured itself.**

What you are asked

There is one kind of installation. What varies is how the machine can be reached afterwards, and that is three tick boxes on one page.

The boxes appear only on a **first** install. An upgrade shows no tasks page at all: the machine already carries the answers, and every one of these settings has a command that changes it afterwards.

1. SYSTEM INTEGRATION

	DEFAULT
Add SD Core to the system PATH, so <code>sd</code> runs from any directory	ticked
Changed afterwards with <code>append.sd.path on</code> or <code>off</code> .	

2. THE SSH SERVER

Installing an ssh server is optional, and the box is not ticked. It downloads from Windows Update and can take several minutes — up to about an hour on a slow machine or connection — which is why it is offered rather than done.

What you see depends on what the machine already has:

MACHINE	BOXES	DEFAULTS
No ssh server	<i>Install the OpenSSH server, and indented under it <i>Let other computers connect</i></i>	both unticked
Server present, firewall shut	<i>Let other computers connect</i>	unticked
Server present, firewall open	<i>Let other computers connect</i>	ticked

The second box is a child of the first, so Windows greys it out until the parent is ticked. You cannot ask for remote ssh access without an ssh server.

The last row matters on a machine that already uses ssh: the box shows the **current** firewall scope, so leaving it alone changes nothing in either direction. `OpenSSH-Server-In-TCP` is Windows' own shared rule and not SD's, so defaulting it to unticked and applying that would silently loopback-lock the ssh a site already relies on.

Changed afterwards with `ssh.server install` or `remove`, and `remote.ssh on` or `off`.

3. THE API

	DEFAULT
Provide the SD Core API (port 4243)	unticked

Let other computers on your network reach it

unticked

The API is off unless you ask for it. Decline it and SD is installed with a configuration carrying no `APIPORT` line, so no socket is opened at all — not a listener behind a closed firewall, but no listener.

Again the second box is a child of the first. The pair appears only when there is no existing configuration file to read the answer from.

Changed afterwards with `remote.api on`, `local` or `off`.

AN INSTALLATION WITH NEITHER IS A SUPPORTED CHOICE

Leaving every remote box unticked gives a working SD that nothing outside the machine can reach. That is a real deployment rather than a degraded one.

Be aware of what it means for accounts. Accounts SD creates are denied the Windows console and Remote Desktop, so they reach SD over ssh or over the API and nothing else — including at this keyboard, where an ssh account arrives by `ssh localhost`. With no ssh server on the machine, `create.account ... ssh` and `... both` are refused, because the account could sign in nowhere. `api` and `none` still work, and `none` is meaningful: an application account reached only by `logto`.

The refusal is tested against the machine when you type the command, not against a decision recorded at install time. Install an ssh server later and `create.account ... ssh` simply starts working.

What lands where

WHAT	WHERE	WAS, ON LINUX
Binaries, and the MSYS2 DLLs beside them	<code>C:\Program Files\SD\usr\bin\</code>	<code>/usr/local/bin</code>
The changelog	<code>C:\Program Files\SD\changelog</code>	—
Configuration	<code>C:\ProgramData\SD\sd.conf</code>	<code>/etc/sd.conf</code>
The SDSYS account	<code>C:\ProgramData\SD\sdsys\</code>	<code>/usr/local/sdsys</code>
User accounts	<code>C:\ProgramData\SD\user_accounts\</code>	<code>/home/sd/user_accounts</code>
Group accounts	<code>C:\ProgramData\SD\group_accounts\</code>	<code>/home/sd/group_accounts</code>
POSIX shared memory	<code>C:\ProgramData\SD\shm\</code>	<code>/dev/shm</code>

`usr\bin` is **load-bearing, not tidiness**. Shipping `msys-2.0.dll` beside the executable relocates the POSIX root to the DLL's directory minus two components, so only that depth puts `/` on `C:\Program Files\SD\`. Do not move the binaries.

The DLLs ship beside `sd.exe` on purpose. Do not change their location.

CONFIGURATION

Server and client both read `SD_CONFIG`, then fall back to `%ProgramData%\SD\sd.conf`.

`sd.conf` is installed `onlyifdoesntexist` and marked never to uninstall, so your edits survive both upgrade and removal.

What the installer creates on the machine

<code>sdusers</code>	grants access to the files under <code>C:\ProgramData\SD</code> . Everyone who uses SD needs it
<code>sdsshonly</code>	carries the two deny rights that confine an account to ssh. Every non-administrator account SD creates joins it
<code>sdu_<name></code>	one per account, created by <code>create.account</code>
the service	String Database (SD)
ACLs	inheritance is broken on <code>C:\ProgramData\SD</code> and access granted narrowly. This is what makes the database private from the rest of the machine

You must sign out and back in after being added to `sdusers`. Windows group membership is carried in your logon token. Until you get a new one, you cannot read the data tree at all, and the symptom looks like a broken install.

OpenSSH

The OpenSSH server is installed only if you ask for it, and the box is not ticked. This section covers what happens when you do ask.

Accounts SD creates cannot log in to Windows, so they reach the machine over ssh or through an API client. An installation offering neither is usable by nobody but you — which is a legitimate choice, and worth making knowingly. A local-only machine is served by `ssh localhost`, which needs no network.

It is slow and it looks like a hang. `Add-WindowsCapability` hands off to `TiWorker` and can work for minutes with nothing on screen. **Never kill it** — interrupting Windows servicing mid-flight is how the component store gets corrupted. It may also leave a reboot pending, which is real; SD itself needs none.

If it cannot be installed, the install still succeeds. It is a Features on Demand capability and can be blocked by policy, a WSUS with no source, a metered connection or an offline machine. The installer reports it in as many words, with the command to retry — and you should read that report, because in that state **no account but yours can sign in anywhere**.

It is the same command either way, and it is repeated here because the closing report is easy to close. From an elevated prompt:

```
powershell -File "C:\Program Files\SD\install-ssh.ps1"
```

It exits **0** installed and running, **2** installed but Windows wants a restart before the service exists, **1** failed — and it prints which, so run it in a window you can read rather than expecting a log. It is safe to run twice: on a machine that already has the server it says so and changes nothing.

The full-screen editors

The installer makes sure the two terminal editors are on the machine, because the `edit` and `micro` verbs run them. Current Windows builds already carry `edit.exe`; `micro` never ships with Windows, so it is always a winget install. Neither is offered as a choice, for the same reason the ssh server is not: a programmer account with a verb that does nothing is worse than either answer.

If one cannot be installed, the install still succeeds. No winget, no network or a policy in the way all end the same: SD is complete and one editor verb is not — `ed`, the line editor, needs nothing. What happened is in `C:\ProgramData\SD\install-editors.log`, and to do it by hand afterwards, from an elevated prompt:

```
winget install -e --id Microsoft.Edit --scope machine
winget install -e --id zyedidia.micro --scope machine
```

The scope is not optional. Without it winget installs into the profile of whoever ran it, and accounts SD creates cannot log in to Windows at all — so a per-user copy is one they can never reach.

Changing any of it afterwards

Every choice on the tasks page has a verb that changes it later, and that is why an upgrade does not ask again. All four need an elevated administrator session, and all four report when given no keyword:

ssh.server install remove	add or remove the OpenSSH server
remote.ssh on off	who may reach it
remote.api on local off	whether SD opens its API socket, and who may reach it
append.sd.path on off	whether <code>sd</code> runs from any directory

They are covered in the SD Core for Windows administrator documentation, under *Remote access and the machine*.

`remote.api on` and `off` change whether SD opens a socket at all, which it decides at start-up, so they offer to restart SD — and that restart ends every session including the one that asked. `local` and `on` differ only in the firewall and take effect at once.

The underlying scripts are still on the machine and can be run directly if you prefer, from an elevated prompt:

```
powershell -File "C:\Program Files\SD\api-firewall.ps1" -Open
```

At the end

The installer **finishes, and then opens SD** so you can set your own password. It does not leave a wizard page waiting behind the session.

A Start Menu entry, **Check the SD installation**, runs a post-install check you can re-run at any time. It closes on a keypress.

You are told plainly what setting no password costs, rather than being allowed to skip past it silently.

Continued in

Upgrading and uninstalling — upgrading an existing installation, and uninstalling.